

双层运输问题

A simple and effective genetic algorithm for the two-stage capacitated facility location problem

1. 问题建模

TSCFLP 是指产品从工厂 (P) 到仓库 (S)，再由仓库到客户 (C) 的两阶段运输。(工厂和仓库都有固定成本)

变量定义：

y_i ：0/1变量，若工厂 i 开启则为1，否则为0。

z_j ：二进制变量，若仓库 j 开启则为1，否则为0。

x_{ij} ：连续变量，从工厂 i 运送到仓库 j 的产品数量。

s_{jk} ：连续变量，从仓库 j 运送到客户 $k \in K$ 的产品数量。

优化目标：

总成本 = 开启仓库的固定成本+开启工厂的固定成本+两个阶段的运输成本

$$\text{Minimize } Z = \sum_{i \in I} f_i y_i + \sum_{j \in J} g_j z_j + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{j \in J} \sum_{k \in K} d_{jk} s_{jk}$$

约束条件：

$$\sum_{j \in J} s_{jk} \geq q_k, \quad \forall k \in K \quad // \text{满足每个顾客需要}$$

$$\sum_{i \in I} x_{ij} \geq \sum_{k \in K} s_{jk}, \quad \forall j \in J \quad // \text{运出仓库的数量要小于运入仓库的数量}$$

$$\sum_{j \in J} x_{ij} \leq b_i y_i, \quad \forall i \in I \quad // \text{对于每个开启的仓库，运出的数量要小于生产的数量}$$

$$\sum_{k \in K} s_{jk} \leq p_j z_j, \quad \forall j \in J \quad // \text{仓库的吞吐量不能超过它的容量}$$

$$x_{ij}, s_{jk} \geq 0$$

$$y_i, z_j \in \{0, 1\} \quad // \text{变量属性}$$

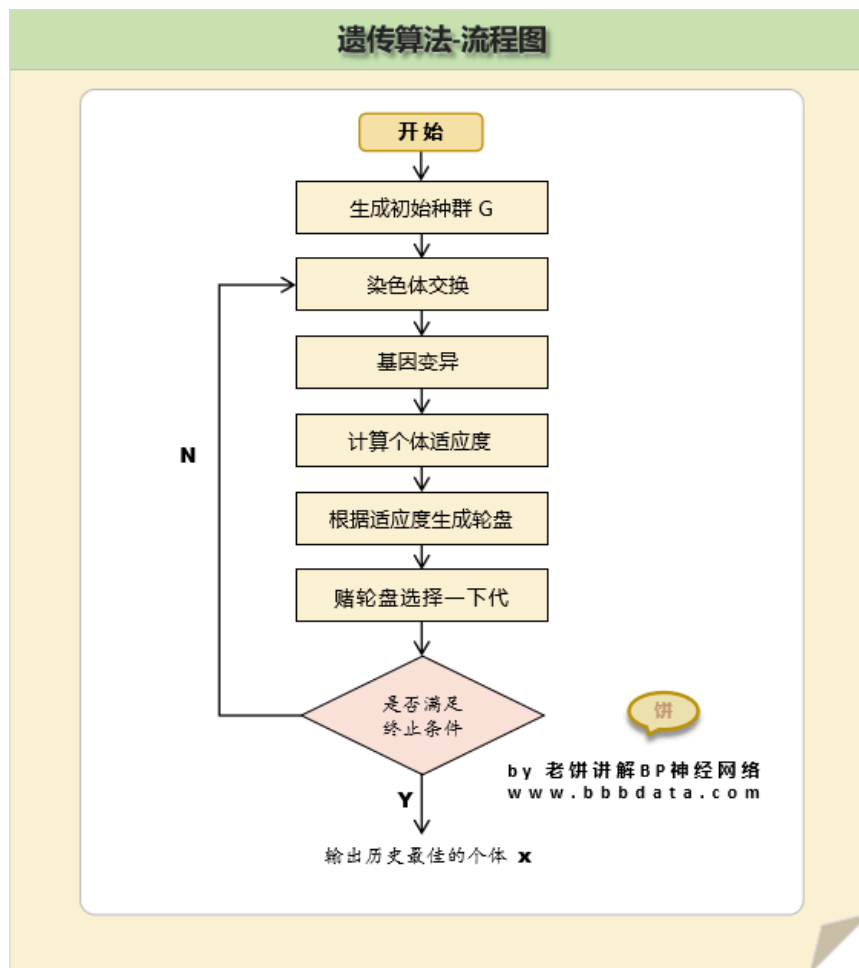
2. 算法设计

核心思想：一旦选址变量确定，剩余的运输变量可以通过最小费用流问题（基于graph的）来获得最优值。所以问题就转化为了如何确定选址，可以采用遗传算法进行求解。

遗传算法的核心在于确定染色体的编码方式：

此处我们设置染色体长度为I+J，即工厂数+仓库数。前I个点位编码第i个工厂是否开启（0不开，1开），后J个点位编码第j个仓库是否开启。

首先看看遗传算法的基本思路（引用一张我觉得画得很清晰的图，详见<https://zhuanlan.zhihu.com/p/667586229>）：



在这个问题的背景下，我们取目标函数的倒数作为fitness function，如果该染色体对应的方案无法满足条件的话，适应度设置为很小的值。

于是我们得到了**算法1**（naive GA）

Algorithm 1. Genetic algorithm (Reeves, 2003)

```
1: Create an initial population of chromosomes
2: while Stopping condition is not satisfied do
3:   repeat
4:     if Crossover condition satisfied then
5:       Select parent chromosomes
6:       Define parameters to crossover
7:       Apply the crossover operator
8:     end if
9:     if Mutation condition satisfied then
10:      Choose mutation points
11:      Apply the mutation operator
12:    end if
13:    Evaluate fitness of offspring
14:  until sufficient offspring created
15:  Select the new population
16: end while
17: return The best individual found
```

这种naive算法缺点有二：

- a. 由于初始化种群质量较低往往需要迭代多次才能找到较好的解（例如初始染色体上大部分位置为0，极少数为1，前期根本满足不了条件，需要大量变异）
- b. 并且容易陷入**局部最优解**（这一点很重要，这解释我们之后在选择初始解的时候概率怎么选择）

在算法一的基础上优化初始解空间，我们可以得到**算法二**

Algorithm 2. Constructive Heuristic – CH₁

```

1:  $y_i \leftarrow 0, \forall i \in I$ 
2:  $z_j \leftarrow 0, \forall j \in J$ 
3: for  $i \in I$  do
4:    $BCP_i \leftarrow \frac{f_i + \sum_{j \in J} c_{ij}}{b_i}$ 
5: end for
6: while  $\sum_{i \in I} b_i y_i \leq \sum_{k \in K} q_k$  do
7:   Select an index  $i' \in \{i | y_i = 0\}$  with probability given by
     
$$\frac{BCP_{i'}}{\sum_{i: y_i = 0} BCP_i}$$

8:    $y_{i'} \leftarrow 1$ 
9: end while
10: for  $j \in J$  do
11:    $BCS_j \leftarrow \frac{\sum_{i \in I} c_{ij} y_i + g_j + \sum_{k \in K} d_{jk}}{b_j}$ 
12: end for
13: while  $\sum_{j \in J} p_j z_j \leq \sum_{k \in K} q_k$  do
14:   Select an index  $j' \in \{j | z_j = 0\}$  with probability given by
     
$$\frac{BCS_{j'}}{\sum_{j: z_j = 0} BCS_j}$$

15:    $z_{j'} \leftarrow 1$ 
16: end while
17: return  $(y, z)$ 

```

定义工厂i的成本效益指数

$$BCP_i = \frac{f_i + \sum_{j \in J} c_{ij}}{b_i}$$

我们按照

$$P(j') = \frac{BCS_{j'}}{\sum_{l|z_l=0} BCS_l} \quad [cite : 408]$$

的概率从未开启的工厂中选择工厂进行开启，知道选择的工厂满足生产容量需求

ps:不难看出这里的概率公式里面成本越高的工厂被选择的概率越高，有效帮助算法跳出局部最优解

在算法二的基础上，我们在增加精英策略和局部搜索提高最终解的质量，得到**算法三**

Algorithm 3. Genetic algorithm with elitism for the TSCFLP

```
1: Create an initial population of chromosomes
2: while Stopping condition is not satisfied do
3:   repeat
4:     if Crossover condition satisfied then
5:       Select parent chromosomes
6:       Define parameters to crossover
7:       Apply the crossover operator
8:     end if
9:     if Mutation condition satisfied then
10:      Choose mutation points
11:      Apply the mutation operator
12:    end if
13:    Evaluate fitness of offspring
14:  until sufficient offspring created
15:  Select the new population
16:  if the best individual found by the algorithm is
    updated then
17:    Perform local searches BL1 followed by BL2 in the
    best individual
18:  end if
19:  Perform local searches BL1 followed by BL2 in the
  whole population every  $N$  generations
20: end while
21: return The best individual found
```

当我们发现了一个局部最优个体我们对于这个最优个体进行一次LS1和LS2的局部搜索

当迭代次数达到N时，我们对于每个个体都执行一次LS1和LS2

LS1：通过翻转单个设施的状态（0变1或1变0）寻找更优解

LS2：通过交换一对设施的状态（一个开变关，另一个关变开）来探索邻域

Formulation and solution of a two-stage capacitated facility location problem with multilevel capacities

1. 问题建模

在论文1的基础上引入仓库不同等级的概念，每个仓库可以选择一个等级 d ，每个等级对应不一样的容量上限和成本

Model A: 每个仓库对每种产品都有独立的存储容量限制和对应的固定成本

Model B: 仓库的总物理容积是有限的，所有产品共享这一容积

新引入的变量： $\hat{f}_{jpd}$: 仓库 j 针对产品 p 选择等级 d 的额外固定成本。

优化目标：

$$\text{Min } Z = \sum_{j \in J} f_j Q_j + \sum_{j \in J} \sum_{p \in P} \sum_{d \in D_{jp}} \hat{f}_{jpd} Y_{jpd} + \sum_{i \in I} \sum_{j \in J} \sum_{p \in P} c_{ijp} X_{ijp} + \sum_{j \in J} \sum_{k \in K} \sum_{p \in P} \hat{c}_{jkp} \hat{X}_{jkp}$$

约束条件：

$$\sum_{j \in J} \hat{X}_{jkp} = r_{kp}, \quad \forall k \in K, p \in P // \text{满足每个用户约束}$$

$$\sum_{i \in I} X_{ijp} = \sum_{k \in K} \hat{X}_{jkp}, \quad \forall j \in J, p \in P // \text{仓库流入流出平衡}$$

$$\sum_{j \in J} X_{ijp} \leq S_{ip}, \quad \forall i \in I, p \in P, S_{ip} : \text{工厂 } i // \text{工厂流入流出平衡}$$

Model A:

$$\sum_{i \in I} X_{ijp} \leq \sum_{d \in D_{jp}} \hat{b}_{jpd} Y_{jpd}, \quad \forall j \in J, p \in P // \text{仓库容量约束}$$

Model B：

$$\sum_{i \in I} \sum_{p \in P} X_{ijp} \cdot \alpha_p \leq \sum_{d \in \hat{D}_j} Y_{jd} \cdot b_{jd}, \quad \forall j \in J // \text{仓库容量约束}$$

新增约束

$$\sum_{d \in D_{jp}} Y_{jpd} \leq Q_j, \quad \forall j \in J, p \in P // \text{只有仓库开启才会有等级}$$

2. 求解算法

MAAT数学启发式算法，核心思想是先降维得到较为优秀的方案，再在原先的维度上精确求解，降低计算量。
这个方法从始至终都是使用的单纯形法等传统优化方法，但是通过聚合有效降低了前期求解难度

(a) Stage 1：基于聚合的迭代搜索

Step1：确定参数 ρ 和 μ 。

ρ 表示填满所有客户需求最少需要多少个仓库。

μ 是一个观察窗口，我们设置聚合完后仓库数量的上限是 $\rho+\mu$

model A：

$$\rho = \max_{p \in P} \left\{ \left\lceil \frac{\sum_{k \in K} r_{kp}}{\text{avg}(\hat{b}_{jpd})} \right\rceil \right\}$$

model B：

$$\rho = \left\lceil \frac{\sum_{p \in P} (\alpha_p \cdot \sum_{k \in K} r_{kp})}{\text{avg}(b_{jd})} \right\rceil$$

Step 2：候选点聚合（核心步骤）。

从全部候选点中挑出不超过 $\rho+\mu$ 个点进行求解

这些点一部分是上一轮迭代发现的“好点”，剩下的从 m 中随机抽取。

Step 3：

对于候选点集合对应的子问题进行求解，此时求解压力小很多

Step 4：

重复迭代T轮，找到最佳选址

(b) Stage 2：局部搜索

尝试交换，把当前解中的一个已开启仓库 j_{in} 关闭，转而开启一个原本关闭的仓库 j_{out} 。

每次交换后评估成本是升了还是降了，这里使用对偶单纯形法求解计算量较小。

保留优化目标下降的交换。

(c) Stage 3：最终优化

回到原先的维度，不再限制仓库数目，进行优化。

可以看到此时的解已经比较优秀了，所以在此基础上进行优化计算量会小很多

对比实验

遗传算法超参数设置：

population size=40

generation=150

crossover rate=0.8

mutation rate=0.05

每迭代10次执行一次局部搜索

Matheuristic超参数：

J=40，聚合后μ=10

outer_iteration=15
局部搜索swap次数为20次

中等规模：
I=20, J=40, K=80

此时遗传算法能够得到和Gurobi算法完全一致的结果，但是耗时较久，Matheuristic算法的结果不是理论最优，但是差距不大。

```
Instance: medium_run0 (I=20, J=40, K=80, D=794)
Gurobi: 10833.000000, time=3.8654s
GA: 10833.000000, time=571.1558s, GAP=0.0000%
Matheuristic: 11034.000000, time=33.5755s, GAP=1.8554%
```

我们发现在问题规模较小时，遗传算法耗时较多（虽然引入了早停机制），所以遗传算法主要是在问题规模较大时具有优势。
ps：近年来Gurobi计算能力突飞猛进，导致复现现有算法基本不可能超过Gurobi计算。

双层VRP问题

A two-echelon location routing problem with mobile satellite

1. 问题背景

- 一级中心 (Primary Depot)：货物的源头
 - 二级仓库 (CD: City Depot)：绑定小车（CF），小车每天从这里领第一批货出发，最后回到这里。
 - 整合点 (CP: Consolidation Point)：临时交接点，不存货，只供大小车交接
 - 一级车辆 (CT: Mobile Satellite)：大车。它从一级中心运货，在 CP 点给小车补货。
 - 二级车辆 (CF: Second-echelon Freightier)：末端小配送车。它们从 CD 出发，中途在 CP 补货，最终完成所有客户配送。
- 一些前置假设：
- 小车只能补货一次，一个用户最终只能由一个小车配送，小车最后要回到出发点，配送费和路程成正比

2. 问题建模

变量定义：
 H_s, H_p, H_v, H_w : 分别为工厂（CD）、中转站（CP）、一级车（CT）和二级车（CF）的固定成本。
 c_{ij}^e : 在 echelon e （1或2）中经过弧 (i, j) 的行驶成本。
 y : 设施是否开启或车辆是否使用的 0-1 变量。
 x_{ij} : 车辆是否经过弧 (i, j) 的 0-1 变量。

优化目标：

$$\min Z = \sum_{s \in S} H_s y_s + \sum_{p \in P} H_p y_p + \sum_{v \in V} H_v y_v + \sum_{w \in W} H_w y_w + \sum_{(i,j) \in A^1} \sum_{v \in V} c_{ij}^1 x_{ij}^v + \sum_{(i,j) \in A^2} \sum_{w \in W} c_{ij}^2 x_{ij}^w$$

约束条件：

第一类约束：时间约束

$$a_j^2 \leq b_j^1 \quad \forall j \in S \cup P$$

$$a_j^1 \leq b_j^1 \quad \forall j \in N^1$$

小车到达 j 点的时间 (a_j^2) 必须早于大车离开该点的时间 (b_j^1) 。

第二类约束：容量约束

$$\sum_{w \in W} f_j^w \leq D_j \quad \forall j \in S \cup P \quad (13)$$

$$D_j \leq \sum_{i \in N^1} \sum_{v \in V} F_{ij}^v \quad \forall j \in S \cup P \quad (15)$$

交界货物的时候小车拿走的货物数小于大车装载的，大于小车最终给到用户的

第三类约束：

$$\sum_{j \in N^2} x_{sj}^w = y_s^w: \text{ 小车必须从指派的二级仓库。}$$

$$\sum_{i \in N^1} x_{ij}^v = \sum_{k \in N^1} x_{jk}^v: \text{ 进入 CP 点的大车必须离开。}$$

3. 问题求解

总体思路：先定下来小车的配送路径，再反推大车的配送路径

(a) Stage 1：确定小车的配送路径

Step 1：

适应度比例选择 (FPS)：对于每一个候选二级仓库 s ，计算其适应度 $f_s = 1/H_s$ ，按照 $P_s = f_s / \sum_{s \in S} f_s$ 的概率选择。固定成本越低的仓库，被选中的概率越高。

Step 2：

利用最邻近启发式算法为客户分配车辆并生成路径。
从 CD 出发，每次寻找距离最近且满足载荷约束的客户，直到所有客户都被覆盖。

Step 3：

同步邻域搜索 (SNS) 对初始路径进行改进，使用 Self-swap（自身路径内交换）、Self-insert（自身路径内插入）、Peer-swap（不同路径间交换）更新路径。

(b) Stage 2：一级路径与 CP 选址优化

亮点：不再盲目尝试所有 CP 点，而是通过聚类来定位

Algorithm 1 Our proposed CSNS
1: while $it = 0 : it_{max}$ do 2: if $it < 1$ then 3: Select CD location 4: Generate 2^{nd} echelon routes 5: end if 6: while $h = 1 : h_{max}$ do 7: if $g < g_{max}$ then 8: Optimize 2^{nd} echelon routes 9: $g = g + 1$ 10: else 11: Optimize CD locations 12: $g = 0$ 13: $h = h + 1$ 14: end if 15: end while 16: Select CP locations 17: Generate 1st echelon routes 18: if the optimized solution is better than the current-best then 19: Replace the current-best with the optimized solution 20: end if 21: $it = it + 1$ 22: end while

Step 1：确定补货需求：

沿着第一阶段生成的路径扫描。当小车服务的客户总需求超过其单次载重上限 Q_w 时，记录其补货前后的坐标。

Step 2：CP 点聚类选址 (K-means)：

使用 K-means 算法 对这些补货坐标点进行聚类，找到 k 个聚类中心，将这些中心点映射到地理位置上距离最近的候选 CP 点。
最小化：

$$\min \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

确定每个小车去哪里补货

Step 3：一级路径生成 (CT Routing)：

选定 CP 点后，大车从一级中心 (Primary Depot)* 出发，将这些 CP 点作为访问目标，规划一条类似 TSP（旅行商问题）的环形路径。

这一步就转化为了旅行商问题。

Step 4：时间同步校验：

计算大车和小车到达 CP 的具体时间。如果 $a_j^2 > b_j^1$ （即大车走早了或到晚了），算法会通过插入等待时间或调整大车出发时刻来强行修复时间同步。

The two-echelon location-routing problem A comparative analy

要点：这篇论文不是提出了一种新的求解算法，而是从建模层面简化了模型，进行了去车标化。每个客户不需要知道这个货物是那一辆小车/大车配送的，它只需要知道这个货物来在哪一个卫星站就行

问题建模

核心决策变量：

- z_p, y_s : 布尔变量，若开启平台 p 或卫星站 s 则为 1。
- x_{ij}^1, x_{ij}^2 : 布尔变量，若第一或第二阶段使用弧 (i, j) 则为 1。
- v_{js} : 布尔变量，若客户 $j \in \mathcal{C}$ 由卫星站 $s \in \mathcal{S}$ 配送则为 1（这是 CF3 的核心变量）

优化目标：

$$\begin{aligned} \min \quad & \sum_{p \in \mathcal{P}} f_p z_p + \sum_{s \in \mathcal{S}} f_s y_s \\ & + F^1 \sum_{p \in \mathcal{P}} \sum_{s \in \mathcal{S}} x_{ps}^1 + F^2 \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{C}} x_{sj}^2 \\ & + \sum_{(i,j) \in \mathcal{A}^1} c_{ij}^1 x_{ij}^1 + \sum_{(i,j) \in \mathcal{A}^2} c_{ij}^2 x_{ij}^2 \end{aligned}$$

约束条件：

- a. 第一阶段路径（工厂到卫星站）
 $\sum_{i \in \mathcal{P} \cup \mathcal{S}} x_{is}^1 = \sum_{j \in \mathcal{P} \cup \mathcal{S}} x_{sj}^1 \leq 1, \quad \forall s \in \mathcal{S}$ （卫星站流量平衡）
 $\sum_{s \in \mathcal{S}} x_{ps}^1 = \sum_{s \in \mathcal{S}} x_{sp}^1 \leq K^1 z_p, \quad \forall p \in \mathcal{P}$ （平台出入平衡与开启关联）
- b. 第二阶段路径（卫星站到用户）
 $\sum_{i \in \mathcal{S} \cup \mathcal{C}} x_{ij}^2 = 1, \quad \forall j \in \mathcal{C}$ （每个客户必须被访问一次）
 $\sum_{i \in \mathcal{S} \cup \mathcal{C}} x_{ij}^2 = \sum_{k \in \mathcal{S} \cup \mathcal{C}} x_{jk}^2, \quad \forall j \in \mathcal{C}$ （客户节点流量平衡）
 $\sum_{j \in \mathcal{C}} x_{sj}^2 = \sum_{j \in \mathcal{C}} x_{js}^2 \leq K^2 y_s, \quad \forall s \in \mathcal{S}$ （卫星站出入平衡与开启关联）
- c. 卫星站分配与容量约束
 $\sum_{s \in \mathcal{S}} v_{js} = 1, \quad \forall j \in \mathcal{C}$ （每个客户仅分配给一个卫星站）
 $x_{sj}^2 + x_{ji}^2 + v_{is} \leq 1 + v_{js}, \quad \forall s \in \mathcal{S}, j, i \in \mathcal{C}$ （CF3 特有的路径与卫星站一致性约束，确保路径属于对应卫星站）
 $\sum_{j \in \mathcal{C}} d_j v_{js} \leq T_s y_s, \quad \forall s \in \mathcal{S}$ （卫星站处理能力限制）
 $\sum_{j \in \mathcal{C}} d_j v_{js} \leq Q^1 \sum_{i \in \mathcal{P} \cup \mathcal{S}} x_{is}^1, \quad \forall s \in \mathcal{S}$ （第一阶段车辆运力限制）
- d. 子回路消除约束
 $u_j - u_i + |C| x_{ij}^2 \leq |C| - 1, \quad \forall i, j \in \mathcal{C}$ （其中 u_i 为访问顺序变量）//防止出现二阶路径不经过卫星站的短路情况

Key：CF3特有约束

如果有一辆车从卫星站 s 直接去了客户 j ($x_{sj}^2 = 1$)，且随后从 j 去了 i ($x_{ji}^2 = 1$)，那么根据这个不等式，客户 i 必须也分配给卫星站 s （即 v_{is} 必须为 1）。
得：
 $x_{sj}^2 + x_{ji}^2 + v_{is} \leq 1 + v_{js}, \quad \forall s \in \mathcal{S}, j, i \in \mathcal{C}$
把整条路径上的所有客户都锁死在同一个起始卫星站上。

2. 问题求解
传统的求解方法

对比实验结果

我们选择了中等规模和较大规模两个场景来对比这两种方式

首先设定一些超参数：

工厂开启固定成本：500±μ

卫星站开启成本：50±μ

大车固定成本：200

小车固定成本：60

大车容量：200

小车容量：60

M50中等规模：

我们设定50个用户，5个工厂和5个卫星站

两种方式求解结果如下：

```
=====
实例：M50（50客户）
=====

CSNS (Sutrisno & Yang, 2023):
  总成本：2711.48
  成本标准差：0.00
  最优/最差：2711.48 / 2711.48
  运行时间：1.17 秒
  可行解：是

CF3 + VIs (Senna et al., 2026):
  总成本：1850.56
  运行时间：22.93 秒
  可行解：是
```

M100较大规模：

我们设定100个用户，8个工厂，8个卫星站

两种方式求解结果如下：

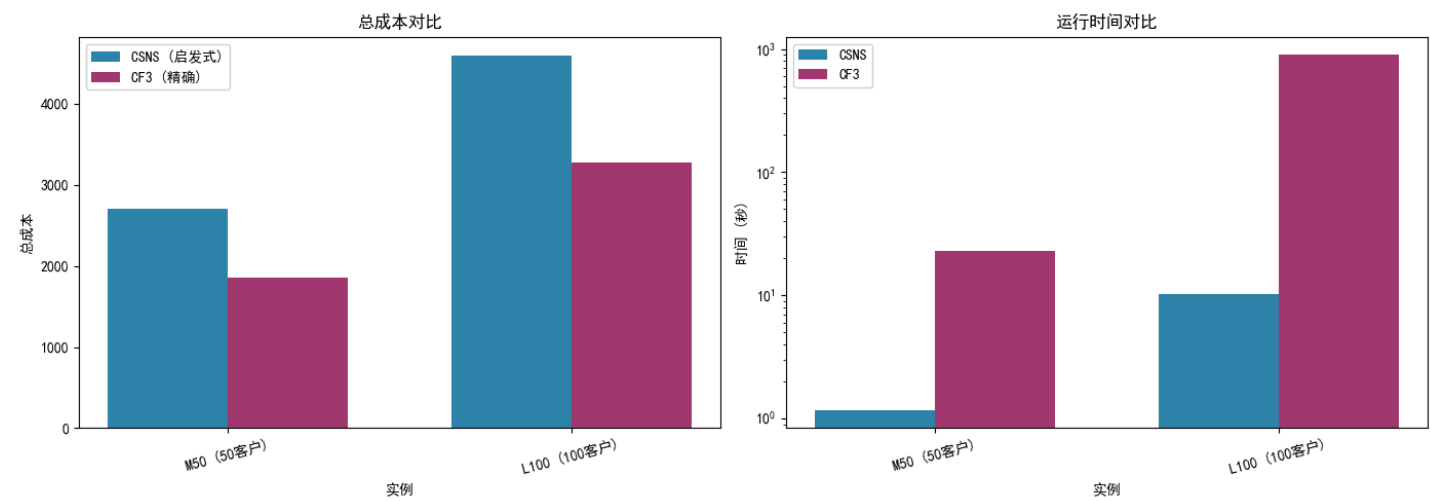
```
=====
实例：L100（100客户）
=====

CSNS (Sutrisno & Yang, 2023):
  总成本：4597.25
  成本标准差：102.56
  最优/最差：4452.21 / 4669.77
  运行时间：10.21 秒
  可行解：是

CF3 + VIs (Senna et al., 2026):
  总成本：3282.08
  运行时间：900.98 秒
  可行解：是

=====
```

对比两种方法我们发现，CSNS方法能够快速求出近似解，CF3方法追求精确解，最终求出的解比CSNS更优但是耗时较长。所以在中等规模任务上，我们可以为了追求更有的解牺牲一下效率；在较大规模任务上，必须考虑到CF3算法的计算成本，合理选择方法。



补充仿真实验

写在前面：我之前的仿真实验全部是拿随机生成的小规模数据实现的，后来有了老师给的仿真数据，于是重新跑了一遍。

双层运输问题

之前GA我的实现方式是直接拿python实现的，当数据集扩大到100-200-300这种做法会很慢，一天的时间只能迭代10轮。在仔细研读论文后，我改成了用c++依赖lemon库的优化器进行复现，得到结果如下表

方法	数据规模 (I-J-K)	总成本 (Objective / Best total cost)	固定成本	运输成本	运行时间 (s)
gurobi	100-200-400	1,452,505.8446	910,006.6700	542,499.1746	14.2972
gurobi	200-400-800	2,855,184.2867	1,772,209.2600	1,082,975.0267	122.4055
GA	100-200-400	1,454,827.8728	912,537.5100	542,290.3628	2712.7171
GA	200-400-800	3,326,625.8629	2,244,163.37	1082462.4929	12777.1424

有趣的是，在小规模数据上GA可以做到完全和gurobi精确解保持一致，而在较大规模数据上，则可以看到启发式算法的不准确性。

```
(.venv) (base) lyaa@Fermi:/mnt/lyaa/2stagetransport$ python tools/maat_solver.py data/csv_100200400
Stage1: customers 400 -> pseudo-customers 80
Set parameter LicenseID to value 2825295
Stage2: runtime 1.58s, |J_open|=47
Stage4: TotalCost=1464859.2585, Gap=0.9190%, Runtime=1.02s (Total 2.76s)
(.venv) (base) lyaa@Fermi:/mnt/lyaa/2stagetransport$ python tools/maat_solver.py data/csv_200400800
Stage1: customers 800 -> pseudo-customers 120
Set parameter LicenseID to value 2825295
Stage2: runtime 5.61s, |J_open|=91
Stage4: TotalCost=2881686.7853, Gap=0.9728%, Runtime=3.89s (Total 9.90s)
```

MAT算法依旧高效，和精确解的差距不足1%，在较大规模的数据上表现卓越。

双层VPR问题

方法	instance_name	best_objective_val	mip_gap	nodes_explored	facility_cost	total_routing_cost	runtime_seconds
CF3	coord20-5-1-2e	78555.00	0.000000	148255	21158.00	57397.00	148.4946
Gurobi (exact)	coord20-5-1-2e	83468.00	0.238235	1	25549.00	57919.00	2.5128
CSNS	coord20-5-1-2e	102219.06			36390.00	65829.06	0.2350

我们发现CSNS时间上竟然打败了Gurobi，确实具有很强优越性，但是代价就是离最优解距离比较大。CF3算法依然比较准确，但是确实比较耗时间。